FoodGO — Project Proposal

Course: CS 488 AI-First Software Engineering, Fall 2026, MUST University **Team:** Hamza Ben Labiedh, Ali Chamakh, Mohamed Krid **Repository:** https://github.com/HamzaBL04/foodGO **Version:** 1.0, 1 November 2026

1. Problem

In Tunisian business districts and campuses, everyone orders lunch in the same 90 minutes. Restaurants are swamped, customers wait, and kitchens cook without knowing how much will be sold. Delivery is improvised by whoever is free.

FoodGO moves ordering to the morning. A customer pre-orders before a deadline the restaurant sets. At the deadline the restaurant knows exactly what to cook, and its **own** delivery staff bring lunch to the customer's office or campus at noon. The platform coordinates; the restaurant delivers.

2. Objectives (business goals from the specification)

ID	Goal	How FoodGO meets it
BG-01, BG-02	Reduce lunch-hour waiting, move ordering before lunch	Morning pre-orders with a hard restaurant deadline
BG-03, BG-04	Help restaurants predict demand and plan preparation	Exact order count at the deadline **plus** an AI-assisted forecast of the next service
BG-05	Organise delivery	Assignment to the restaurant's own delivery people, tracked to confirmation
BG-06	Convenient payment	Cash on delivery or e-payment
BG-07	Restaurants restrict access by their own rule	In-person registration and per-restaurant eligibility
BG-08, BG-09	Order accuracy and visibility	Digital orders, live status for customer, restaurant and courier

3. Users

Customers (employees and students), restaurant staff, the restaurant's delivery people, and the platform administrator. External systems: Tunisian e-payment providers, SMS/OTP, notifications, and optionally an LLM.

4. Scope

In scope for the first release: in-person customer registration and SMS-code sign-in, menus and availability, morning pre-ordering with deadline, customer verification, delivery assignment and

confirmation, cash and e-payment status, live order tracking, administration, and an advisory AI demand forecast.

Out of scope until stakeholders decide (spec §19.7): cancellation, refunds, failed-delivery handling, a specific real payment provider, GPS tracking, delivery fees, self-service registration, a platform-owned delivery fleet.

5. Key user stories and acceptance criteria

The full prioritised backlog (31 stories, MoSCoW, story points) is in `docs/requirements/BACKLOG.md`. The core stories:

Story	Acceptance criteria
US-01 As restaurant staff I register a customer in person.	Name, phone and organization recorded; identity checked in person; duplicates refused.
US-02 As a customer I sign in with an SMS code.	Single-use 6-digit code, 5-minute expiry, attempts limited, unregistered numbers refused.
US-11 As a customer I order before the deadline and get an order number.	Accepted before the deadline, refused after it (ORDER-001); unique FG-XXXXXX number; no duplicate on retry.
US-12 As restaurant staff I see new orders immediately.	Live board of my restaurant's orders only, with customer verification info.
US-16 As restaurant staff I assign an order to my own delivery person.	Only my own couriers; assignment before dispatch; no double assignment.
US-18 As a delivery person I start and confirm the delivery.	Only the assignee can confirm, only after starting; cash collection recorded.
US-19 As a customer I follow my order live.	Status changes appear without refreshing; I only see my own orders.
US-24 As restaurant staff I get a prep forecast for the next service.	Expected quantity, range and confidence per dish; accuracy shown; advice only.

6. Methodology: Scrum with one-week sprints

Why Scrum and not Kanban. The course imposes fixed, dated milestones (requirements, design, midterm MVP, final). Scrum's time-boxed sprints map onto those dates, and the sprint review is a natural checkpoint before each submission. The team is small (three registered members), so a lightweight Scrum fits: one Product Owner owns the specification and backlog, and the implementer pulls work from the sprint backlog. In practice the Sprint 1 implementation was carried out by one member (Hamza Ben Labiedh) working with an AI coding agent; see §7 and the AI usage log. Kanban's continuous flow suits work that arrives unpredictably, like support tickets; our scope is known and our risk is the deadline, which Scrum makes visible.

Why one-week sprints instead of the usual two. Milestones are one to two weeks apart, and the remaining scope after the midterm is small and well understood. Short sprints mean feedback from

the instructor reaches the backlog quickly.

Ceremonies (planned from Sprint 2): sprint planning at the start of each sprint, a short async stand-up in the team chat, review and retrospective at the end. Sprint 1 was a single intensive working session, so its planning was the written plan at the start of that session and its review was the pre-submission verification. **Definition of done:** code merged through a pull request, CI green (lint and tests, coverage ≥ 60 %), acceptance criteria met, docs updated, AI use logged.

AI in the process. AI drafted the first user stories and estimates from the specification; the team validated and corrected them (record at the end of `BACKLOG.md`). AI also produced first drafts of code, diagrams and documentation. Every change went through a pull request gated by CI (lint, tests, coverage); in Sprint 1 the pull requests were self-merged after CI passed, without a second reviewer. Peer review of each PR by another member is planned from Sprint 2. See `docs/ai/AI_USAGE_LOG.md`.

7. Release plan

Sprint	Dates	Goal	Status
0	14 Sep – 28 Sep	Requirements elicitation, specification (Lab 3), software design (Lab 4)	Done
1	1 Nov	Midterm MVP in one intensive session: full pre-order flow for all four roles, AI forecast, CI, documentation, first refactoring and performance pass with metrics	Done
2	2 Nov – 8 Nov	Peer review of the Sprint 1 code by all members, accessibility audit, deployment	Planned
3	9 Nov – 15 Nov	Stakeholder decisions on unresolved items (cancellation, failed delivery), real notifications	Planned
4	16 Nov – final	Hardening, final presentation, defence	Planned

8. Architecture and technology

Following the SDD: a **layered** single deployable (presentation → business components → data access), **client–server**, MVC confined to the presentation layer, **events** only for notification fan-out. Patterns: State (order lifecycle), Strategy (payment methods), Adapter (payment, SMS, notifications, LLM), Observer (live updates).

Stack: Node.js 22 LTS, Express 5, SQLite (better-sqlite3), zod validation, plain HTML/ES-module frontend, node:test + supertest, ESLint, GitHub Actions. Chosen because it is boring and runs anywhere with one `npm install`; nothing needs an external service to work.

9. Risks

Risk	Mitigation
Unresolved business rules (cancellation, refunds, failed delivery)	Not invented; State pattern leaves one extension point per decision
No real payment provider selected (Q-22)	Sandbox adapter; a provider is one new adapter
Over-reliance on AI-generated code	CI gate on every PR (lint, tests, ≥ 60 % coverage), AI usage log, peer review from Sprint 2, team code walkthrough before the defence
Data privacy of customers	Restaurant-scoped queries, minimal data on the live stream, no personal data sent to the LLM